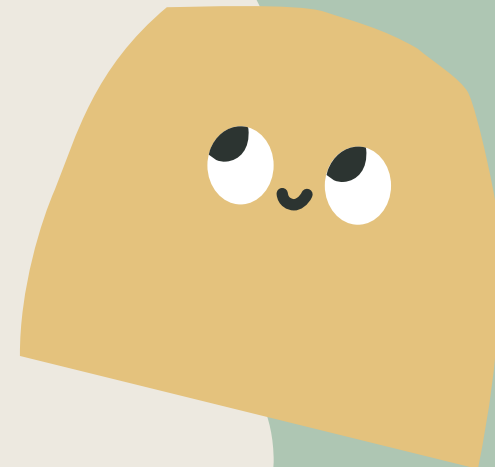
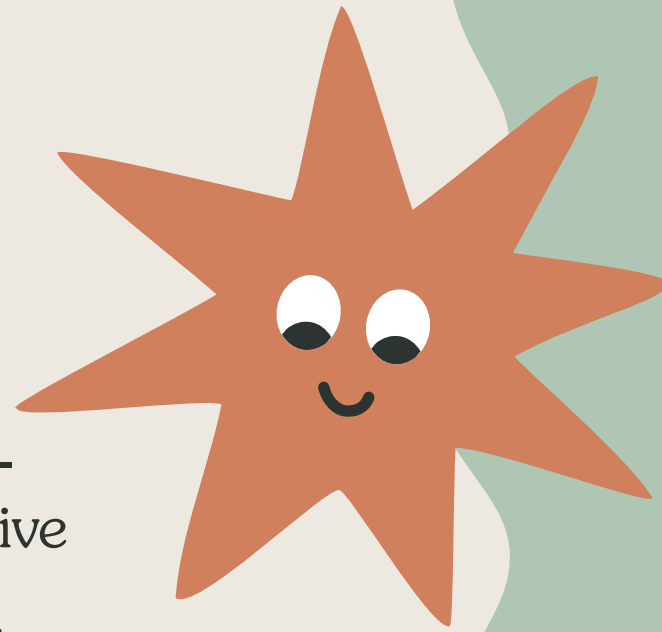


Healthcare AI Billing Assistant

Healthcare billing today is slow, manual, and error-prone — leading to delayed claims and administrative overhead. This project introduces an AI-powered billing assistant that automates key billing tasks, reduces human error, and improves claim accuracy — built using a full-stack architecture spanning database, backend, AI integration, and frontend.



Project Problem and Goal

problem

Manual billing relies on repetitive data entry and manual claim checks — steps that are easy to get wrong. Small errors like missing insurance details or incorrect codes lead to rejected claims, payment delays, and extra rework for staff.

Project Goal

Build an AI-powered billing assistant that automates data entry, flags likely errors before claims are submitted, and lets staff query billing data conversationally — reducing manual work and improving claim accuracy.

Tools and Setup

To build this project, I used SQL Server and SSMS for the database, Python for data ingestion, Node.js and Express for the backend API, and React for the frontend interface.

The first step was designing the database schema in SQL Server — starting with core tables for patients, providers, and claims — then verifying the structure in SSMS before moving on to backend development.

SQL

```
-- 1. Create the Database
CREATE DATABASE HealthcareBillingDB;
GO

USE HealthcareBillingDB;
GO

-- 2. Create Patients Table
CREATE TABLE Patients (
    patient_id INT IDENTITY(1,1) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    dob DATE NOT NULL,
    insurance_id VARCHAR(50) NOT NULL,
    insurance_name VARCHAR(100) NOT NULL
);

-- 3. Create Providers Table
CREATE TABLE Providers (
```

Create Main Project Folder

I created a single main project folder — **healthcare-ai-billing** — on the desktop to house the full project. This folder would contain the backend, frontend, and data ingestion scripts, keeping the database, API, and UI components organized in one place as the project grew.

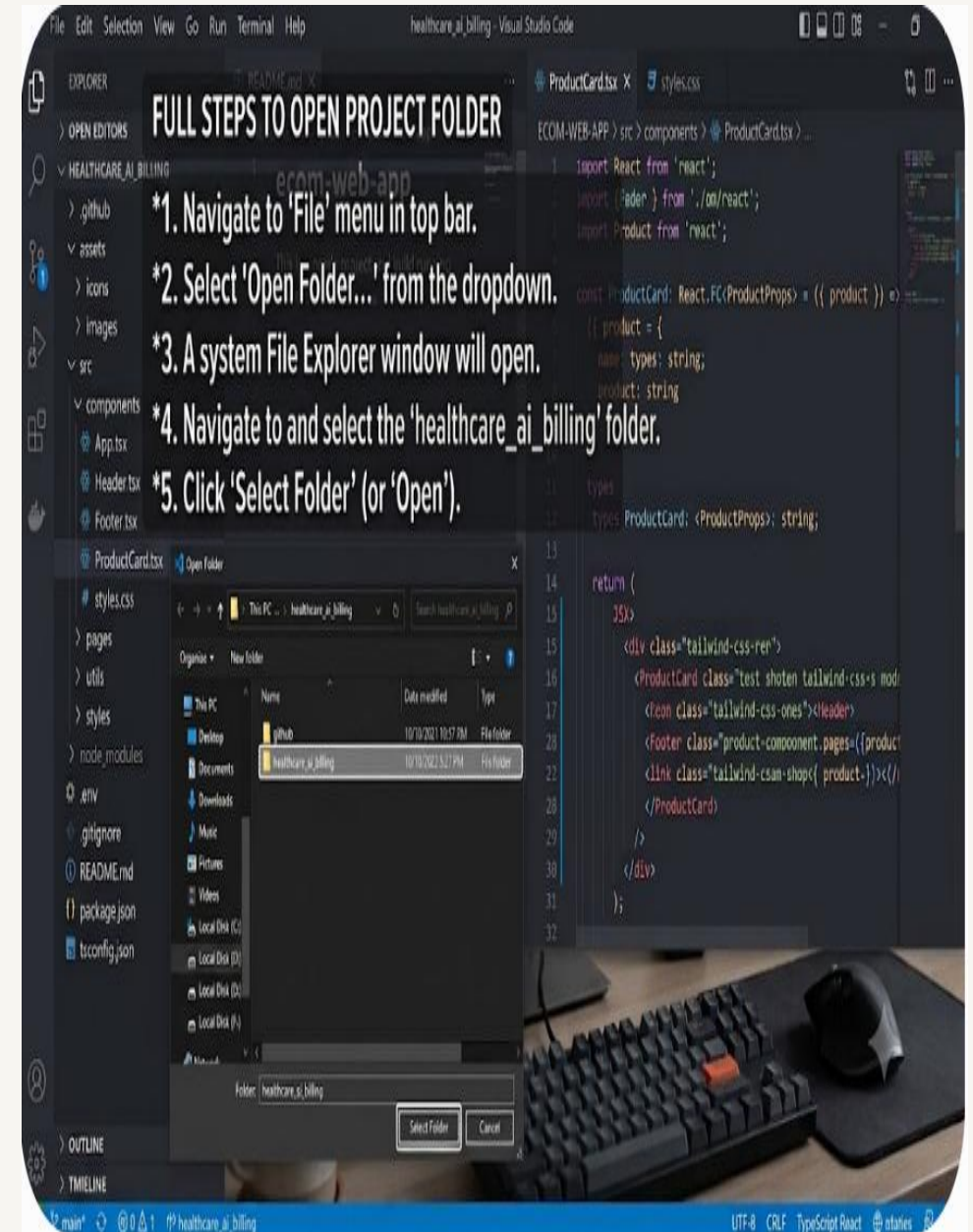


Open Folder in VS Code

With the project folder created, I opened Visual Studio Code and used File → Open Folder to select it, as shown below. This gave me a workspace to organize the project files and begin coding.

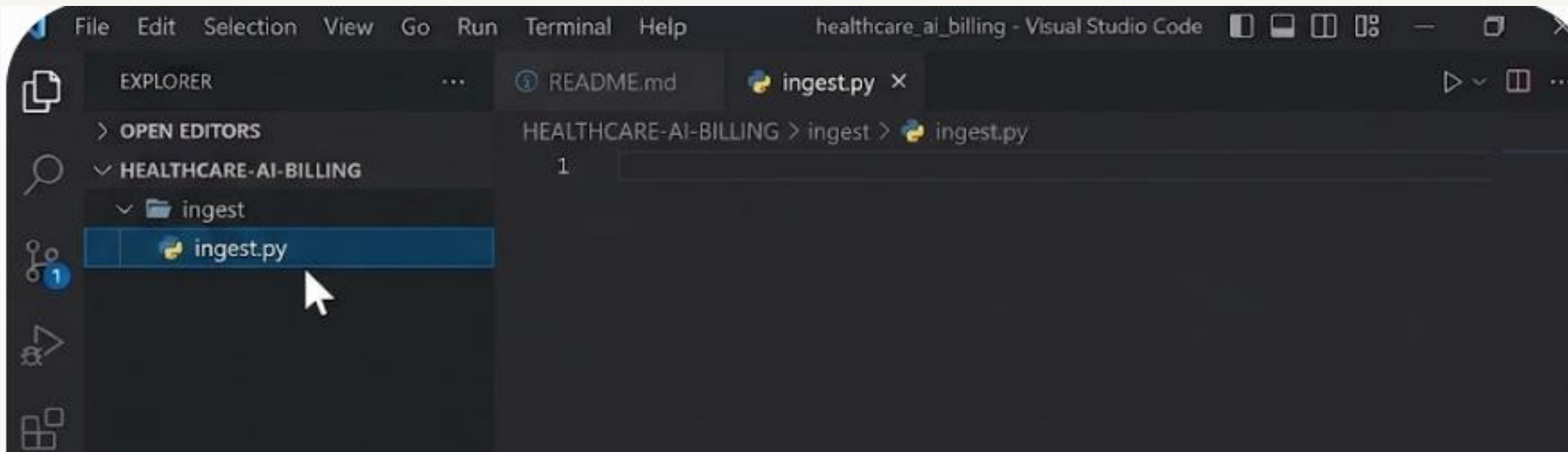
FULL STEPS TO OPEN PROJECT FOLDER

- Navigate to "File" in the top bar.
- Select "Open Folder..." from the dropdown.
- A system File Explorer window will open.
- Navigate to and select the `healthcare-ai-billing` folder.
- Click "Select Folder" (or "Open").



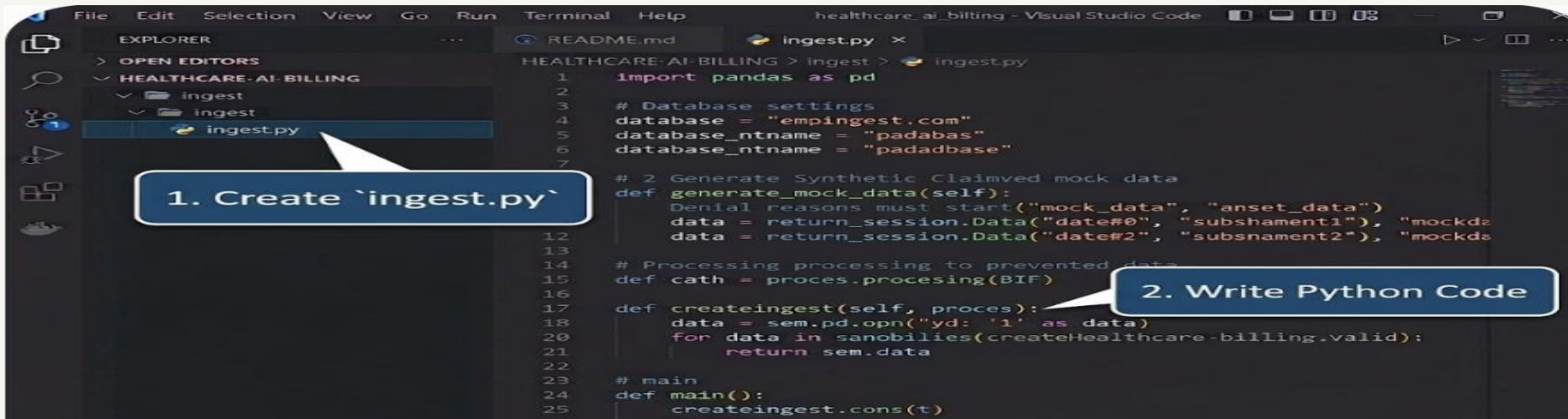
Create ingest.py

Inside the project folder, I created a Python file named **ingest.py** to handle loading data into SQL Server. This script would generate and insert mock patient, provider, and claims data, giving me realistic sample data to test the rest of the application.



Add Python Code

I added the Python script into `ingest.py` to connect to SQL Server, generate the mock patient, provider, and claims data, and insert those records into the database. This handled all the data-loading logic and prepared the project for verification in SSMS in the next step.



```
1  import pandas as pd
2
3  # Database settings
4  database = "empingest.com"
5  database_ntname = "padabas"
6  database_ntname = "padabase"
7
8  # 2 Generate Synthetic Claimved mock data
9  def generate_mock_data(self):
10     Denial reasons must start("mock_data", "anaset_data")
11     data = return_session.Data("date#0", "subshament1"), "mockdz
12     data = return_session.Data("date#2", "subsnament2"), "mockda
13
14  # Processing processing to prevented data
15  def cath = proces.processing(BIF)
16
17  def createingest(self, proces):
18     data = sem.pd.opn("yd: '1' as data)
19     for data in sanobilies(createHealthcare-billing.valid):
20         return sem.data
21
22  # main
23  def main():
24     createingest.cons(t)
```

Verify Data in SSMS

After running `ingest.py`, I opened SSMS and ran a SELECT query on the Patients, Providers, and Claims tables. The results, shown below, confirm that the mock data was inserted correctly and the database was ready for the next stage of development.

```
USE HealthcareBillingDB;

-- Check total row count and review a sample of 10 claims
SELECT COUNT(*) AS TotalClaims FROM Claims;

SELECT TOP 10
    claim_id,
    claim_number,
    status,
    total_amount,
    paid_amount,
    denial_reason,
    submitted_date
FROM Claims;
```

150 %

Results

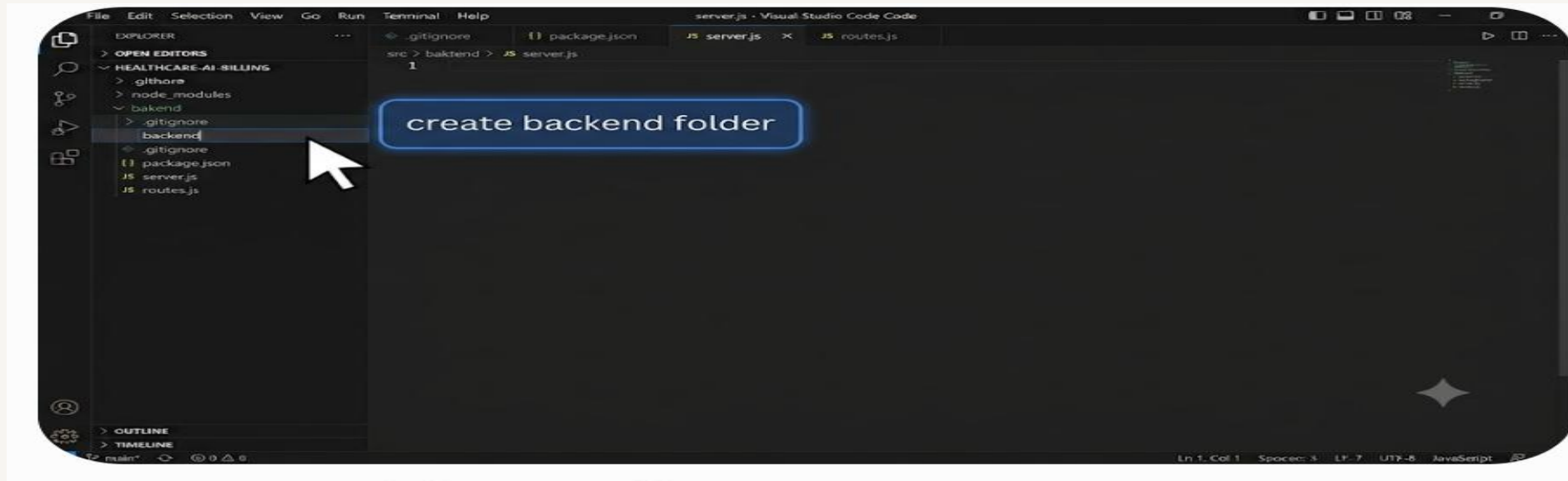
Messages

TotalClaims	
1	100

	claim_id	claim_number	status	total_amount	paid_amount	denial_reason	submitted_date
1	1	CLM-2026-0001	Paid	3219.73	3219.73	N/A	2026-06-03
2	2	CLM-2026-0002	Paid	1413.28	1413.28	N/A	2026-07-21
3	3	CLM-2026-0003	Paid	3295.48	3295.48	N/A	2026-05-25
4	4	CLM-2026-0004	Paid	1828.73	1828.73	N/A	2026-07-10
5	5	CLM-2026-0005	Pending	945.70	0.00	N/A	2026-05-01
6	6	CLM-2026-0006	Paid	2415.07	2415.07	N/A	2026-05-04
7	7	CLM-2026-0007	Paid	1305.12	1305.12	N/A	2026-06-20
8	8	CLM-2026-0008	Denied	884.20	0.00	Prior authorization required	2026-07-20
9	9	CLM-2026-0009	Paid	294.10	294.10	N/A	2026-06-06
10	10	CLM-2026-0010	Paid	3160.19	3160.19	N/A	2026-07-09

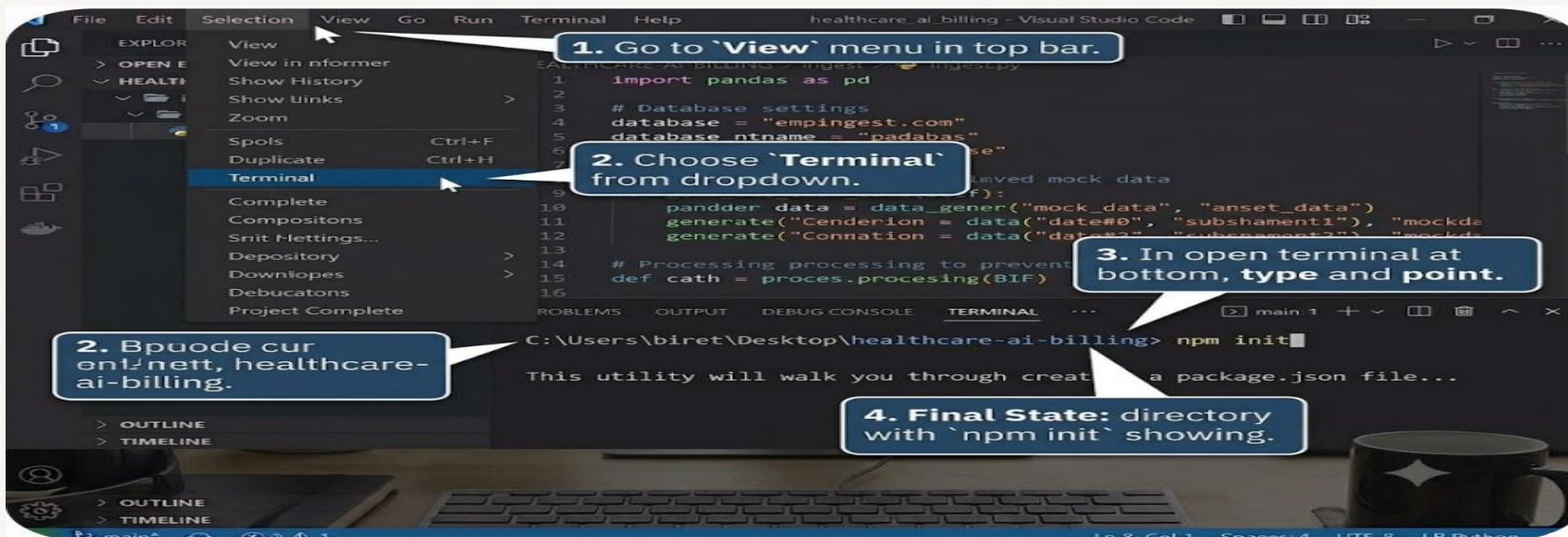
Create Backend Folder

Inside the main project folder, I created a backend folder to hold all server-side code, separate from the database scripts and the future frontend. This kept the project organized as it grew, with each part of the stack — database, backend, and frontend — living in its own clearly defined space.



Initialize Node.js Project

With the **backend** folder created, I opened the terminal in VS Code, navigated into it, and ran **npm init** to **initialize the Node.js project**. This generated the **package.json** file, giving me a clean way to manage backend dependencies going forward.



Install Backend Dependencies

After running npm init to create package.json, I ran npm install to add the required backend packages — including Express and its supporting dependencies. This created the node_modules folder shown below, along with package-lock.json and .package-lock.json, which lock the exact versions of every installed package. With these dependencies in place, the backend was ready for the server code in the next step.

```
backend/  
├─ node_modules/      (installed Node/Express dependencies)  
├─ .package-lock.json  
├─ package-lock.json  
└─ package.json
```

Backend Server Setup

Create and Start Express Server

Set up and launch server.js

Inside the backend folder, I created server.js as the main entry point for the Express application to handle server setup, middleware configuration, and API routes. I then ran `node server.js` in the terminal to start the backend and confirmed it was running correctly and ready to handle incoming API requests.

```
backend/  
├── node_modules/      (installed Node/Express dependencies)  
├── .package-lock.json  
├── package-lock.json  
├── package.json  
└── server.js          (Backend server entry point)
```

Store API Key Securely

API keys private in .env.

To protect sensitive information, I stored the API key in a secure .env file inside the backend folder — never the frontend, since frontend variables are exposed to the browser. I also made sure to exclude the key from screenshots and slides. This is an important best practice for application security and backend development.

```
backend/
├── node_modules/    <-- Installed dependencies (Express, dotenv, etc.)
├── .env             <-- Environment variables & secret API keys
├── .gitignore       <-- Specifies files to untrack in Git (includes .env)
├── package-lock.json <-- Exact dependency tree lockfile
├── package.json     <-- Project metadata & scripts
└── server.js        <-- Main Express application entry point
```


What Accomplished

Backend foundation completed.

At this stage, the backend foundation was complete. The project had a working Node.js and Express server, secure API key handling, and the structure needed to move forward with AI integration.

- ✓ Backend folder created
- ✓ Node.js project initialized (package.json)
- ✓ Dependencies installed (Express + supporting packages)
- ✓ server.js created and running
- ✓ API key stored securely in .env

Review Project Structure and Prepare Backend Setup

Review Project Structure

After completing the backend foundation, I reviewed the overall project layout to confirm that the database scripts, backend files, and frontend folder were each organized clearly and in the right place — keeping the project easy to navigate as it grew in complexity.

Prepare Backend Setup

Prepare the backend for API development.

With the folder structure confirmed, I made sure the Express backend was fully organized — server file, dependencies, and environment configuration all in place — and ready for building the AI chat endpoint in the next step.

```
healthcare-ai-billing/  
├── backend/  
│   ├── node_modules/  
│   ├── .env      (API key stored securely, server-side only)  
│   ├── .gitignore (excludes .env from Git)  
│   ├── .package-lock.json  
│   ├── package-lock.json  
│   ├── package.json  
│   └── server.js  
├── frontend/      (folder created, setup pending)  
└── ingest.py
```

AI Endpoint Integration & API Testing

To enable communication between the client and the model, I implemented an Express AI route (`/api/v1/chat`) to receive incoming user prompts, securely pass them to the AI service using environment variables, and return structured model responses back to the application. The endpoint was verified using curl and Postman to confirm reliable request handling, proper error handling, and the expected payload structure before connecting the React frontend.

Create the AI Analyzer Script (ai_analyzer.py)

Build the standalone Python AI engine.

In addition to the chat endpoint, I created ai_analyzer.py at the project root as a standalone script for deeper billing analysis. It reads its own environment variables from .env, sends healthcare billing data to the AI API, and parses the response into structured medical insights — giving the project a second, independent path for AI-driven analysis separate from the real-time chat feature.

Python

```
# ai_analyzer.py - Core AI Generation Logic
load_dotenv()
client = OpenAI(base_url="https://api.groq.com/openai/v1", api_key=os.getenv("GROQ_API_KEY"))

def generate_ai_appeal(claim_row):
    prompt = f"Analyze denied claim #{claim_row['claim_number']}..."
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content
```

How the AI Analyzer Works

To automate the evaluation process, ai_analyzer.py queries SQL Server using pandas to extract denied claims (SELECT * FROM Claims WHERE status = 'Denied'), dynamically mapping each claim's number, billed amount, and denial reason into a structured LLM prompt. These record-specific prompts are sent to Groq's llama-3.3-70b-versatile model, which transforms raw denial data into clear root-cause summaries and formal appeal letters ready for payer submission.

Express API Bridge & Python Subprocess Execution

To connect the frontend to the AI engine, I created an Express route (`/api/generate-appeal`) that executes `ai_analyzer.py` as a backend subprocess. When a request comes in for a denied claim, Node.js triggers the Python script, captures the JSON payload it returns — containing the root cause and appeal draft — and sends that response back to the client as the HTTP response.

Project & Backend Structure Verification

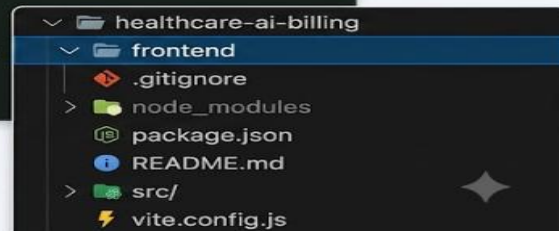
Before scaling API features and frontend components, I audited the project structure to confirm strict separation between the database scripts, Express backend files, Python AI modules, and future React components. This verification ensured the Node.js server was securely configured and properly organized — ready for further API route development, reliable script execution, and eventual frontend integration.

Set Up React Frontend

Create the frontend with Vite.

After completing the backend and AI integration, I created the frontend using React and Vite to build a fast, responsive user interface. This gave me a modern development setup for building the dashboard and connecting it to the backend services.

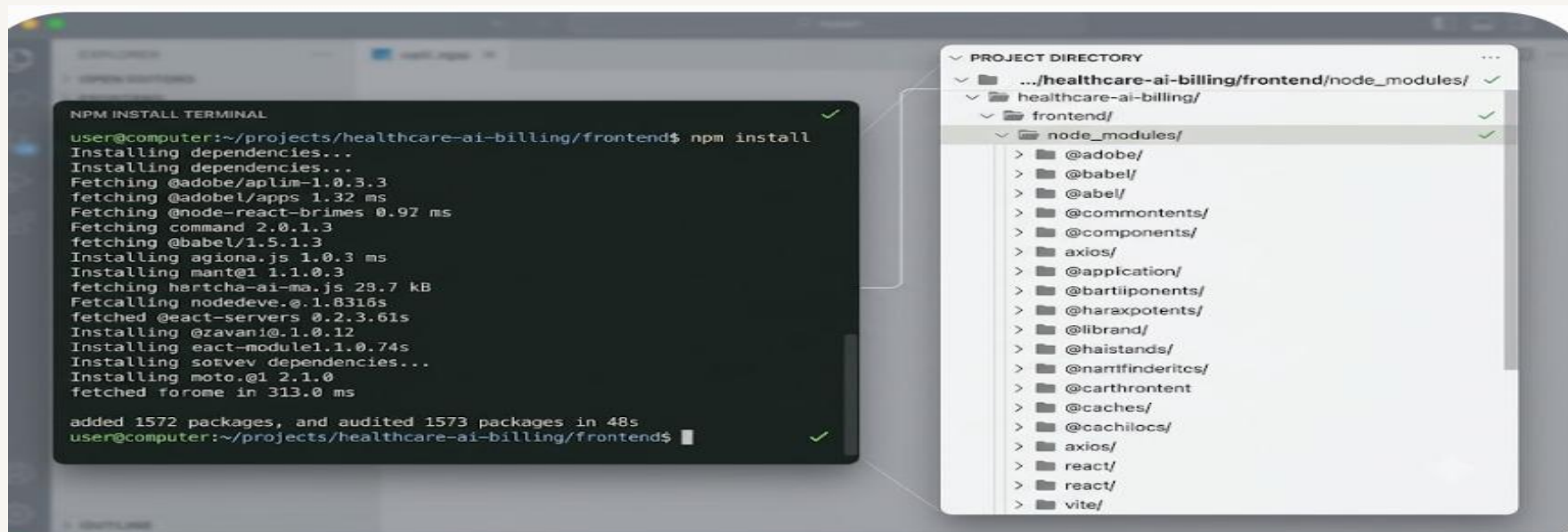
```
user@computer:~/projects/healthcare-ai-billing/  
> npm create vite@latest frontend -- --template react  
Need to install the following packages: create-vite  
  
Ok to proceed? (y)  
...  
Scaffolding project in /path/to/healthcare-ai-billing/frontend...  
  
Done. Now run:  
  cd frontend  
  npm install  
  npm run dev
```



Install Frontend Packages

Install dependencies.

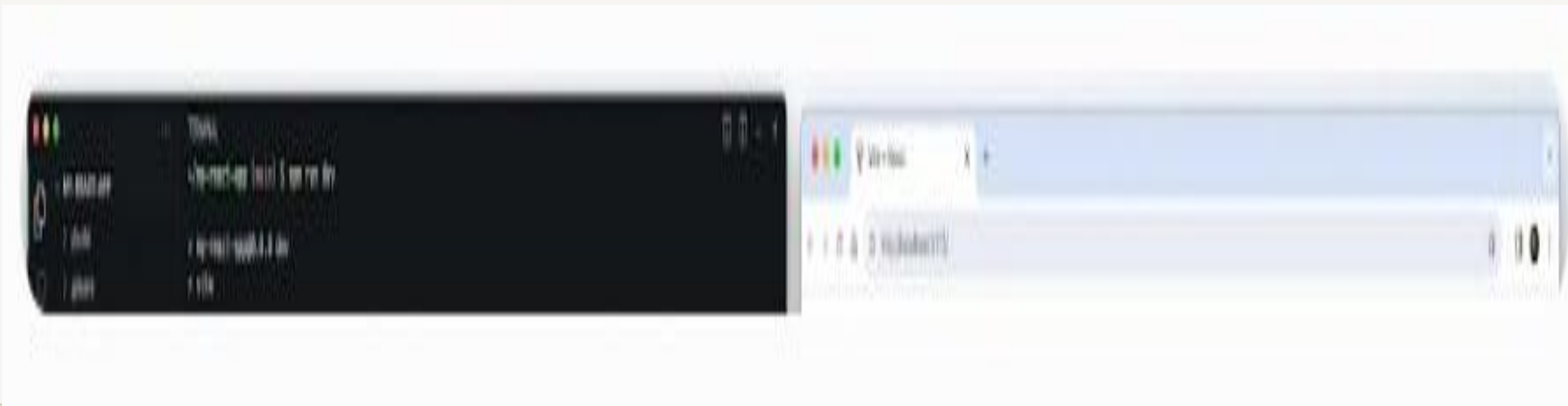
I installed the required frontend dependencies so the React application could run properly and support the components needed for the user interface. This set up the project environment for building and testing the frontend.



Start Frontend Server

Run the Vite development server.

After installing the packages, I started the Vite development server to confirm the React application launched successfully — verifying the frontend setup was working correctly before adding more components.



Review Project Structure and Configure Proxy

Review Project Structure

Check folders and files.

I reviewed the overall project structure to confirm the backend and frontend files were organized properly, with each part of the application in a clear, logical place before continuing development.

Configure Proxy — Update vite.config.js

Point /api to localhost:5000.

To support smooth communication between the frontend and backend during development, I configured a proxy in vite.config.js so any request sent to /api would be forwarded to the Express backend running on localhost:5000. This avoided cross-origin issues and made local testing seamless.

Create Chat Component, Build Chat Logic, and Render Chat in App.jsx

Create Chat Component

Create Chat.jsx.

I created a React component called Chat.jsx to handle the chat interface for the application. This component manages the user prompt input, loading state, and communication with the backend API.

Build Chat Logic

Add state and fetch logic.

Inside the chat component, I added React state and fetch logic to send user prompts to the backend and receive the AI response. This made the interface interactive and allowed the application to display responses dynamically on the screen.

Render Chat in App.jsx

Import and display Chat.

After building the chat component, I imported it into App.jsx and rendered it in the main application layout. This made the chat interface visible in the UI and allowed users to interact with the AI feature directly.

Show Data Table

Display records in a table.

I displayed the records in a structured table so users could easily review the available data in one place. The table layout makes it simple to scan records and supports future actions such as viewing details, editing entries, or deleting records if needed.

Test Full-Stack Integration

Test frontend and backend together.

After completing both the frontend and backend setup, I ran the application and tested the full connection between the two layers. This allowed me to confirm that the React interface could successfully communicate with the Express backend and display data as expected.

Final Verification

Review output and fix errors.

As a final step, I reviewed both the terminal output and the browser results to make sure the application was running as expected. If any errors appeared, I identified and corrected them to confirm the project was stable and ready for presentation.

Final Dashboard Output

A complete view of the working application.

The dashboard below shows the finished Healthcare AI Billing Assistant, pulling live claims data from SQL Server into a structured table with claim ID, payer, billed amount, and claim status. This brings together the database, backend, and frontend work covered throughout this project into one connected, working interface.

 **Healthcare AI Billing Assistant**

Llama-3.3-70b + SQL Server Integration

Refresh Claims

CLAIM ID	PATIENT	PAYER	BILLED AMT	STATUS	ACTIONS
CLM-2026-0001	N/A	Medicare / Private	\$3,219.73	Paid	Generate AI Appeal
CLM-2026-0002	N/A	Medicare / Private	\$1,413.28	Paid	Generate AI Appeal
CLM-2026-0003	N/A	Medicare / Private	\$3,295.48	Paid	Generate AI Appeal
CLM-2026-0004	N/A	Medicare / Private	\$1,828.73	Paid	Generate AI Appeal
CLM-2026-0005	N/A	Medicare / Private	\$945.7	Pending	Generate AI Appeal
CLM-2026-0006	N/A	Medicare / Private	\$2,415.07	Paid	Generate AI Appeal
CLM-2026-0007	N/A	Medicare / Private	\$1,305.12	Paid	Generate AI Appeal
CLM-2026-0008	N/A	Medicare / Private	\$884.2	Denied	Generate AI Appeal
CLM-2026-0009	N/A	Medicare / Private	\$294.1	Paid	Generate AI Appeal

Conclusion

Final project complete.

This project brings together conversational AI, automated LLM-driven claim analysis, database management, backend development, and frontend design to create a practical healthcare billing solution. It demonstrates how these tools can work together to reduce manual billing work, surface root causes for denied claims, and generate appeal-ready documentation — improving both efficiency and decision-making in healthcare billing operations.